

Observability Co-Pilot

AI-Powered Infrastructure Intelligence

Technical Overview & Architecture Document

Version 2.0

Full-Stack Observability Platform

FastAPI | React + Vite | SQLite | WebSocket | Welford's Algorithm

Generated for project documentation

Table of Contents

- 1. Executive Summary
- 2. System Architecture
- 3. Backend Engine
- 4. Anomaly Detection Pipeline
- 5. Root Cause Analysis (RCA)
- 6. Real-Time Communication
- 7. Frontend Dashboard
- 8. API Reference
- 9. Database Schema
- 10. Running the Project
- 11. Technology Stack Summary

1. Executive Summary

The Observability Co-Pilot is an autonomous telemetry platform that monitors distributed microservices in real time. It ingests metrics, logs, and traces from five simulated microservices, detects anomalies using statistical methods (Z-score and IQR), and automatically generates Root Cause Analysis (RCA) reports with actionable remediation steps.

The system operates as a self-contained demo: a built-in telemetry simulator generates realistic metric streams, periodically injects anomalies, and the detection pipeline processes them end-to-end - from raw data to structured incident reports pushed to the dashboard via WebSocket in real time.

Key Capabilities

- Real-time metric ingestion from 5 microservices (api-gateway, user-service, order-service, payment-service, inventory-service)
- Statistical anomaly detection using Welford's online algorithm ($O(1)$ per data point) and IQR fencing
- Automated Root Cause Analysis with contributing factors and remediation steps
- Live dashboard with WebSocket-powered incident streaming
- Incident lifecycle management: detect, analyse, rate, resolve
- Configurable per-service alert thresholds (Z-score, IQR, cooldown, direction sensitivity)
- Dead Letter Queue (DLQ) for failed event processing

2. System Architecture

The system follows a two-tier architecture: a Python/FastAPI backend serving as both the data engine and API layer, and a React/Vite frontend providing the real-time dashboard. Communication flows through REST APIs for data queries and WebSocket for live incident push.

Architecture Overview

The data pipeline flows as follows:

- Telemetry Simulator generates metrics, logs, and traces every 2 seconds for 5 services
- Metrics are stored in SQLite and simultaneously fed to the Detection Engine
- The Detection Engine applies Z-score (continuous metrics) or IQR (error rates) analysis
- CRITICAL anomalies trigger the Correlation/RCA Engine, which assembles telemetry context
- RCA generates structured incident reports (root cause, factors, remediation steps)
- Incidents are stored in the database and broadcast to all WebSocket clients
- The React dashboard polls for data and receives live pushes via WebSocket

Data Flow Diagram

Simulator --> Metrics DB --> Detection Engine --> Anomaly Events --> RCA Engine --> Incidents --> WebSocket --> Dashboard

Component Breakdown

Component	Technology	Responsibility
Backend API	FastAPI / Python	REST endpoints, WebSocket, lifecycle management
Database	SQLite +aiosqlite	Persistent storage for all telemetry and incidents
Simulator	Python asyncio	Generates realistic telemetry for 5 microservices
Detection	Welford / IQR	Statistical anomaly detection on metric streams
RCA Engine	Template-based	Root cause analysis with context assembly
Frontend	React + Vite + TS	Real-time dashboard with incident management
WebSocket	FastAPI WS	Live incident push to connected dashboard clients

3. Backend Engine

The backend is a single FastAPI application (app/main.py) that bootstraps the entire system on startup: initializes the SQLite database, seeds default configurations, starts the telemetry simulator as a background asyncio task, and exposes 15+ REST endpoints.

Startup Lifecycle

- Database tables are created via init_db() with the full TRD v2 schema (10 tables)
- Default alert configuration is seeded for the '__default__' service profile
- A demo API key ('demo-key-2026') is hashed with bcrypt and stored for auth
- The telemetry simulator starts as an asyncio background task
- WebSocket client list is shared with the simulator for live broadcasting

Telemetry Simulator (simulator.py)

The simulator models 5 realistic microservices, each producing 3 metrics at 2-second intervals. Every ~30 seconds (after 100 seconds of baseline warmup), it injects an anomaly by spiking a metric 4-8x above its standard deviation.

Simulated Services

Service	Monitored Metrics
api-gateway	http_request_duration_ms, throughput_rps, http_error_rate_percent
user-service	http_request_duration_ms, cpu_usage_percent, memory_usage_bytes
order-service	http_request_duration_ms, throughput_rps, cpu_usage_percent
payment-service	http_request_duration_ms, http_error_rate_percent, timeout_count
inventory-service	cpu_usage_percent, memory_usage_bytes, throughput_rps

Beyond metrics, the simulator also generates realistic log events (INFO/WARN/ERROR/FATAL) and distributed trace spans with proper trace_id/span_id correlation. Log templates include messages like 'Connection pool nearing capacity: 85% used' and 'Circuit breaker OPEN for payment-service'.

4. Anomaly Detection Pipeline

The detection engine (detection.py) implements two complementary algorithms, selected automatically based on metric type. It processes every metric event inline during simulation, enabling sub-second detection latency.

4.1 Z-Score Detection (Continuous Metrics)

For continuous metrics (latency, CPU, memory, throughput), the engine uses Welford's online algorithm to maintain a running mean and variance with $O(1)$ time and $O(1)$ memory per data point. This avoids storing historical windows.

- Cold-start protection: alerts are suppressed until $n \geq 100$ data points (~200 seconds of warmup)
- Z-score is computed BEFORE updating the baseline (prevents self-dampening)
- Thresholds: $|z| \geq 2.0$ triggers WARNING, $|z| \geq 3.0$ triggers CRITICAL (configurable)
- Direction sensitivity: can detect upper-only, lower-only, or both directions

Welford's Online Algorithm

The WelfordState class (welford.py) implements the recurrence relation:

```
n += 1
delta = value - mean
mean += delta / n
delta2 = value - mean
M2 += delta * delta2
variance = M2 / n
z_score = (value - mean) / sqrt(variance)
```

4.2 IQR Detection (Error Rate Metrics)

For error-rate metrics (http_error_rate_percent, timeout_count, failed_job_count), the engine uses Interquartile Range (IQR) fencing on a 60-point sliding window.

- Upper fence = $Q3 + 1.5 * IQR$ (triggers WARNING)
- Extreme fence = $Q3 + 3.0 * IQR$ (triggers CRITICAL)
- Minimum 10 data points required before detection activates

4.3 Deduplication & Cooldown

To prevent alert storms, the engine implements two suppression mechanisms:

- Dedup Key: MD5 hash of service:metric:5min_bucket prevents duplicate alerts within 5-minute windows
- Cooldown Timer: after firing a CRITICAL alert, the service enters a configurable cooldown period (default 5 minutes)

5. Root Cause Analysis (RCA)

When a CRITICAL anomaly fires (and is not suppressed), the Correlation Engine (correlation.py) assembles a telemetry context window and generates a structured RCA.

5.1 Context Assembly

The engine collects three categories of telemetry from the anomaly's context window:

- Logs: up to 200 recent log entries from the affected service
- Traces: all spans within the 5-minute context window
- Metrics: 30-minute metric history for trend analysis

This assembled context is stored as an immutable snapshot in the raw_contexts table, preserving the exact data that influenced the analysis.

5.2 RCA Generation

The current implementation uses template-based RCA (designed for LLM upgrade). Templates are indexed by anomaly_type and metric_name, providing domain-specific root causes, contributing factors, and remediation steps.

Example RCA Output (CPU Spike)

Root Cause:

CPU utilisation exceeded safe threshold - triggered by compute-intensive request pattern or thread pool exhaustion

Contributing Factors:

- Spike in concurrent request volume saturating thread pool
- CPU-bound JSON serialisation on large payloads
- Background cron jobs competing for CPU cycles

Remediation Steps:

- Increase CPU resource limits for the container
- Enable request queuing with backpressure
- Offload heavy serialisation to worker threads
- Reschedule background jobs to off-peak hours

Each RCA includes a confidence score (0.0-1.0) indicating analysis reliability. Template-based RCAs score 0.45-0.75; an LLM integration would yield higher scores.

6. Real-Time Communication

6.1 WebSocket Protocol

The backend exposes a WebSocket endpoint at `ws://localhost:8000/ws/incidents`. Clients receive structured JSON messages for real-time incident streaming.

Message Types

Type	Direction	Purpose
connected	Server -> Client	Confirms connection with unique client_id
incident	Server -> Client	New incident with full RCA payload
ping	Server -> Client	Keep-alive every 30 seconds

The frontend implements automatic reconnection with a 3-second delay on disconnect. Connection status is shown in the dashboard header as a live/connecting indicator.

6.2 REST API Polling

In addition to WebSocket push, the dashboard polls REST endpoints at configurable intervals: 5 seconds for incidents/stats/metrics, 10 seconds for services/health. This ensures data consistency even if the WebSocket connection drops.

7. Frontend Dashboard

The frontend is a React + TypeScript application built with Vite, featuring a landing page with 3D animations and a full-featured operational dashboard.

7.1 Landing Page (route: /)

A marketing-style page with six sections: Navigation, Hero (with WebGL nebula and starfield backgrounds), Product Overview, Architecture diagram, Documentation, Enterprise features, and Footer. The Hero section links directly to the dashboard.

7.2 Dashboard (route: /dashboard)

The operational dashboard is organized into three tabs:

Overview Tab

- 4 stat cards: Active Incidents, Total Anomalies, Services Monitored, Avg Confidence
- Incident Feed: scrollable list of recent incidents with severity badges and time-ago labels
- Incident Detail: full RCA view with root cause, signal values (observed/baseline/z-score), contributing factors, remediation steps, star rating, and model info
- Service Cards: grid showing each service's status, incident/anomaly/metric counts (24h)
- Live Metrics: SVG sparkline charts with per-service metric selection and auto-refresh

Anomalies Tab

- Tabular view of all anomaly events with time, service, type, severity, metric, value, z-score
- Auto-refreshes every 5 seconds, capped at 30 most recent events

System Health Tab

- Database, Simulator, and WebSocket health status with detailed check info
- System uptime counter

7.3 UI Components

Component	Description
SeverityBadge	Color-coded badge (red/amber/blue) with pulse animation for CRITICAL
StatusDot	Colored dot indicator for service health (green/amber/red + pulse)
StatCard	Glassmorphism card with gradient background, icon, and large value
ConfidenceBar	Horizontal progress bar color-coded by confidence level
StarRating	Interactive 1-5 star rating for user feedback on RCA quality
Sparkline	SVG-based mini chart with gradient fill and polyline stroke
MetricsPanel	Multi-metric display with service selector and auto-refresh

8. API Reference

All REST endpoints are prefixed with /api/v1 and accept an optional X-API-Key header. Responses follow the envelope format: {status, data, meta}.

Method	Endpoint	Description
GET	/health	System health with component checks and uptime
GET	/stats	Dashboard stats: incidents, anomalies, services, confidence
GET	/incidents	List incidents (filterable by service, severity, time range)
GET	/incidents/:id	Get single incident with full RCA detail
PATCH	/incidents/:id/rating	Rate an incident's RCA quality (1-5 stars)
PATCH	/incidents/:id/resolve	Mark an incident as resolved
GET	/anomalies	List anomaly events (filterable, paginated)
GET	/metrics/:service	Get metrics grouped by name with time-series data
GET	/services	List monitored services with status and 24h counts
GET	/alerts/config	List per-service alert threshold configurations
POST	/alerts/config	Create/update alert config (upsert by service_name)
GET	/dlq	List unreviewed Dead Letter Queue events
PATCH	/dlq/:id/review	Mark a DLQ event as reviewed
GET	/baselines	Debug: view all Welford baseline states

WebSocket Endpoint

```
ws://localhost:8000/ws/incidents
```

Accepts connections and pushes incident payloads in real time. Sends ping every 30s.

9. Database Schema

The system uses SQLite with WAL mode for concurrent read/write. The schema includes 10 tables adapted from the PostgreSQL-based TRD v2 specification.

Table	Rows (approx.)	Purpose
incidents	Grows with alerts	Core incident records with RCA data
anomaly_events	Grows with alerts	All detected anomalies (WARNING + CRITICAL)
raw_contexts	1 per incident	Immutable telemetry snapshots for RCA audit
metrics	Pruned to 2 hours	Time-series metric data points
logs	Pruned to 2 hours	Structured log events with trace correlation
traces	Pruned to 2 hours	Distributed trace spans
alert_configs	1 per service	Per-service detection thresholds
api_keys	1 (demo)	API key authentication (bcrypt hashed)
dlq_events	Minimal	Dead Letter Queue for failed processing
system_health	Periodic	System health snapshots

Data retention: The simulator automatically prunes metrics, logs, and traces older than 2 hours (every 200 seconds) to keep the local database compact.

10. Running the Project

Prerequisites

- Python 3.11+
- Node.js 18+ with npm

Backend Setup

```
cd backend
pip install -r requirements.txt
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

The backend starts, creates the SQLite database, seeds demo data, and begins simulating telemetry. The API is available at <http://localhost:8000/api/v1>.

Frontend Setup

```
cd Frontend/app
npm install
npm run dev
```

The Vite dev server starts at <http://localhost:3000>. Navigate to `/dashboard` for the operational view. Anomaly data begins appearing after ~100 seconds.

Key URLs

URL	Description
http://localhost:3000	Landing page with product marketing
http://localhost:3000/dashboard	Real-time operational dashboard
http://localhost:8000/api/v1/health	Backend health check endpoint
ws://localhost:8000/ws/incidents	WebSocket incident stream

11. Technology Stack Summary

Backend

Package	Version	Role
FastAPI	0.115.0	Async web framework with auto-generated OpenAPI docs
Uvicorn	0.30.6	ASGI server with hot-reload for development
Pydantic	2.9.2	Data validation and serialisation for API models
aiosqlite	0.20.0	Async SQLite driver for non-blocking database I/O
bcrypt	4.2.0	Password/API key hashing
structlog	24.4.0	Structured logging

Frontend

Package	Version	Role
React	19.2.0	UI framework with hooks-based architecture
Vite	7.2.4	Build tool with instant HMR and ESM-native dev server
TypeScript	5.9.3	Type-safe JavaScript with strict mode
Tailwind CSS	3.4.19	Utility-first CSS framework for rapid UI development
React Router	7.6.1	Client-side routing (landing page + dashboard)
GSAP	3.15.0	Animation library for landing page effects
Three.js	0.184.0	3D WebGL for nebula/starfield hero backgrounds
Recharts	2.15.4	Chart library (available for future extensions)
Lucide React	0.562.0	Icon set for UI elements

Production Roadmap

The template-based RCA engine is designed for drop-in LLM replacement. The `raw_contexts` table stores the exact telemetry snapshot that would be sent as an LLM prompt, enabling seamless upgrade to GPT-4, Claude, or Gemini for higher-fidelity root cause analysis. The `confidence_score` field and `llm_model` tracking are already in place for A/B testing.